DL Architectures & Generative Techniques

Master Summary

Stefan Eder

Based on *Deep Learning and Neural Networks* SS25

How to use this document. This is a structured summary built around *running text*. Explanations, derivations, and logical flow between ideas live in the prose — colored boxes are reserved for the few things that must stand out at a glance.

- **Running text** — read for understanding, derivations, and context.
- **Colored boxes** — scan for key definitions, theorems, examples, and exam notes.

Boxes lose their impact when overused. When in doubt, write prose.

Compiled March 14, 2026

Contents

1

Image Classification

Computer vision tasks span a spectrum of difficulty. In *image classification* a single class label is predicted for the most prominent object in an image. *Object detection* additionally localizes each instance with a bounding box, *semantic segmentation* assigns class labels per pixel, and *instance segmentation* combines both. This chapter traces the rapid evolution of CNN-based image classification, from AlexNet's breakthrough in 2012 to transformer-based approaches a decade later. The top-5 error on ImageNet fell from roughly 26% to under 3% in five years, eventually reaching and surpassing the human error rate of $\approx 3.5\%$.

1.1 The ImageNet Benchmark

The ImageNet Large-Scale Visual Recognition Challenge (ILSVRC) became the standard yardstick for image classification progress. Each image is labeled with its dominant object class; the dataset covers 1000 classes. Two evaluation protocols are used: **top-1 error** — the highest-ranked prediction must be correct — and **top-5 error** — the correct class must appear within the five highest-ranked predictions. Most papers report top-5 error, which is less sensitive to label ambiguity.

Before 2012 the leading methods relied on hand-crafted features (SIFT, HoG, Haar-like features); CNNs changed that completely. A key additional observation is that features learned for ImageNet transfer remarkably well to other datasets: pre-training on ImageNet and fine-tuning on a smaller target dataset consistently outperforms training from scratch, especially when the target set is small.

Top-k Error

A prediction is correct under the *top-k criterion* if the true class label appears among the k highest-scoring predictions of the model. Top-5 error is the fraction of test images for which this criterion fails.

1.2 AlexNet (2012)

AlexNet (Krizhevsky et al., 2012) won ILSVRC-2012 with a top-5 error of **15.3%**, far below the runner-up's 26.2%, and marked the start of the deep-learning era in vision.

The network consists of 5 convolutional layers followed by 3 fully-connected layers, totaling 60 million parameters and 650k neurons. The input is a $224 \times 224 \times 3$ image. Training ran for 5–6 days on two GTX 580 3 GB GPUs; GPU memory was the binding constraint.

AlexNet Architecture

- **8 learned layers**: 5 convolutional + 3 fully-connected
- **Parameters**: $\approx 60 \times 10^6$
- **Input**: $224 \times 224 \times 3$
- **Output**: 1000-way softmax

Four innovations drove AlexNet's success. **ReLU activations** replaced tanh, providing substantial speed-up in training. **Dropout** in the fully-connected layers acted as a strong regularizer against overfitting. **Overlapping max-pooling** used 3×3 windows with stride 2. Finally, **Local Response Normalization** (LRN) provided competitive lateral inhibition across neighboring feature maps, loosely inspired by cortical inhibition — though later work showed LRN does not actually help and it was abandoned.

LRN is obsolete

Local Response Normalization was used in AlexNet and ZfNet but shown by VGGNet not to improve performance. Batch Normalization replaced it entirely.

1.3 ZfNet (2013)

ZfNet (Zeiler & Fergus, 2013) achieved **13.5%** top-5 error. Its main contribution was a *deconvolution-based visualization* technique: by reversing the forward pass through convolution, pooling, and ReLU layers, one can project feature maps back to pixel space and inspect what each filter responds to.

This analysis revealed that AlexNet's first-layer 11×11 kernels with stride 4 created aliasing artifacts and many "dead" (never-activated) filters. ZfNet replaced them with 7×7 kernels and stride 2, yielding cleaner low-level features and better accuracy with comparable architecture depth. ZfNet also performed the first systematic *transfer learning* experiments, showing that fine-tuning a pre-trained model on CalTech-101/256 quickly surpassed previous state-of-the-art on those benchmarks.

1.4 VGGNet (2014)

VGGNet (Simonyan & Zisserman, 2014) won the localization task and placed second in classification at ILSVRC-2014 with **7.3%** top-5 error. It was directly motivated by ZfNet's finding that large kernels are suboptimal.

Receptive-Field Equivalence of Stacked 3×3 Kernels

Two stacked 3×3 convolutional layers (stride 1) cover the same receptive field as a single 5×5 layer, and three 3×3 layers equal one 7×7 layer. The stacked design is strictly better: it introduces more non-linearities and has *fewer* parameters ($2 \times 3^2 C^2 = 18C^2$ vs. $5^2 C^2 = 25C^2$ for C channels).

VGGNet uses *exclusively* 3×3 convolutions with stride 1. The two most common variants are **VGG-16** (best single-crop top-5) and **VGG-19** (marginally better with multi-crop evaluation). The study also confirmed that LRN does not help and that 1×1 convolutions alone (without depth increase) do not improve performance.

A serious flaw: the first fully-connected layer takes the $7 \times 7 \times 512$ feature map and maps it to 4096 units *without* spatial weight sharing, creating a single layer with $> 100 \times 10^6$ parameters — more than 80% of the entire network. This makes VGGNet prone to overfitting and expensive to deploy.

1.5 Network in Network (2014)

Network in Network (NiN; Lin et al., 2014) did not enter ILSVRC but introduced two ideas that shaped all subsequent architectures.

MLPconv layers. Replace the linear filter in a standard convolution with a small MLP applied to each local patch. In practice this equals a $k \times k$ convolution followed by two 1×1 convolutions with ReLU — increasing non-linear representational power without enlarging the receptive field.

Global Average Pooling (GAP). Instead of flattening spatial feature maps into a vector and feeding fully-connected layers, the final convolutional layer produces one feature map per class; GAP then reduces each map to a scalar by averaging.

Global Average Pooling

Given a feature tensor of shape $H \times W \times C$, GAP outputs a C-dimensional vector where the c-th entry is $\frac{1}{HW} \sum_{i,j} x_{i,j,c}$. Applied to a C-class classification head this yields *zero* extra parameters while providing spatial translation invariance and making each channel directly interpretable as a class confidence map.

Intuition & Mental Model

GAP can be seen as enforcing the network to make its last feature maps "honest": channel c must actually encode evidence for class c, because the spatial average of that channel is the final logit.

1.6 GoogLeNet / Inception v1 (2014)

GoogLeNet (Szegedy et al., 2014) won ILSVRC-2014 classification with **6.7%** top-5 error using 22 layers and only **6.9 million parameters** — less than 6% of VGGNet's 122 million. The key insight: 85% of AlexNet/VGG parameters lived in the fully-connected layers and mainly contributed to overfitting; replacing them with GAP plus one FC layer saves $> 99\%$ of those parameters.

1.6.1 Inception Modules

The building block is the *Inception module*, which processes the input in parallel through 1×1, 3×3, and 5×5 convolutions as well as a max-pooling branch, then concatenates all outputs along the channel dimension. This lets the network choose its own filter sizes rather than committing to one scale.

The naive version has two problems: (i) the channel dimension grows at every module since pooling cannot reduce channels, and (ii) 5×5 convolutions on a large feature map are expensive. Both are solved by **bottleneck** 1×1 **convolutions**: placed before the 3×3 and 5×5 branches they reduce the input channel count, and placed after the pooling branch they prevent channel explosion.

Bottleneck Parameter Savings (Inception 4a example)

Without bottleneck:

$3{\times}3$ branch : $3^2 \times 480 \times 208 \approx 898$k params; $5{\times}5$ branch : $5^2 \times 480 \times 48 \approx 576$k params.

With 1×1 bottleneck (96 and 16 intermediate channels respectively):

$$3{\times}3 : 480 \times 96 + 3^2 \times 96 \times 208 \approx 225\text{k}; \quad 5{\times}5 : 480 \times 16 + 5^2 \times 16 \times 48 \approx 27\text{k}.$$

Savings of roughly $4\times$ and $20\times$ per branch.

Because the network is 22 layers deep, the gradient signal can weaken before reaching the early layers (vanishing gradient). GoogLeNet addresses this with two **auxiliary loss heads** attached to intermediate layers during training; they are discarded at inference.

1.7 Batch-Normalized Inception / Inception v2 (2015)

The 2015 Inception variant integrated Batch Normalization (BN) after every convolution, achieving **4.8%** top-5 error. The headline result is speed: BN reduced the number of training steps needed to reach 72% accuracy by a factor of **14**. BN also replaced Dropout as the primary regularizer in this architecture. An improved down-sampling block ran max-pooling and a stride-2 bottleneck convolution in parallel, then concatenated the results.

1.8 Highway Networks (2015)

Highway Networks (Srivastava et al., 2015) asked: can gradient descent train networks with *hundreds* of layers? The answer is yes, with *gating* borrowed from LSTMs.

A standard feedforward layer computes $\mathbf{h} = H(\mathbf{x}; \mathbf{W}_H)$. A Highway layer adds learnable transform and carry gates:

Highway Network Layer

$$\mathbf{h} = H(\mathbf{x}; \mathbf{W}_H) \odot T(\mathbf{x}; \mathbf{W}_T) + \mathbf{x} \odot (1 - T(\mathbf{x}; \mathbf{W}_T)),$$

where $T(\mathbf{x}; \mathbf{W}_T) = \sigma(\mathbf{W}_T\mathbf{x} + \mathbf{b}_T)$ is the *transform gate* and $\odot$ is element-wise multiplication. Initializing $\mathbf{b}_T$ to a large negative value biases the network to carry information forward early in training, allowing gradients to flow unimpeded.

100-layer Highway Networks were successfully trained on CIFAR-10 and CIFAR-100, directly motivating residual networks.

1.9 ResNet (2015)

ResNet (He et al., 2015) won every track of ILSVRC-2015, achieving **3.57%** top-5 error on classification — below the human error rate — and won 194 of 200 categories in detection. It became the dominant backbone for downstream vision tasks.

1.9.1 Residual Learning

The central observation: plain networks degrade in accuracy as depth increases beyond a certain point, even on the training set, suggesting an optimization problem rather than overfitting. If a shallower network achieves accuracy A, a deeper one should be able to match it by learning identity mappings in the added layers — yet this does not happen.

The fix: instead of asking k stacked layers to approximate the target function $F(\mathbf{x})$ directly, ask them to approximate the *residual* $F(\mathbf{x}) - \mathbf{x}$, then add the input back via a *skip connection*.

Residual Block

$$\mathbf{h} = \mathcal{F}(\mathbf{x}; \{\mathbf{W}_i\}) + \mathbf{x},$$

where $\mathcal{F}$ is the residual mapping (two or three convolutional layers) and $\mathbf{x}$ is the identity shortcut. *No extra parameters* are added by the shortcut.

Intuition & Mental Model

If the optimal function is close to identity, it is easier to push $\mathcal{F}$ toward zero than to learn an identity from scratch through a stack of non-linear layers. Gradients can also flow directly through the shortcut to lower layers, mitigating vanishing gradients.

1.9.2 Bottleneck Blocks

For ResNet-50 and deeper, the basic $3 \times 3/3 \times 3$ block is replaced by a *bottleneck* block: 1×1 (reduce channels) $\rightarrow$ 3×3 $\rightarrow$ 1×1 (restore channels). For example, ResNet-50 processes $256 \rightarrow 64 \rightarrow 64 \rightarrow 256$ channels per block. This keeps computation comparable to ResNet-34 while tripling depth.

1.9.3 Projection Shortcuts

When a skip connection crosses a change in spatial resolution or channel count (e.g., between stages), the identity shortcut is replaced by a 1×1 convolution with stride 2, matching dimensions for the addition.

1.9.4 Variants and FLOPs

Variant	Parameters	FLOPs
ResNet-18	11.7×10^6	1.8×10^9
ResNet-34	21.8×10^6	3.6×10^9
ResNet-50	25.6×10^6	3.8×10^9
ResNet-101	44.5×10^6	7.6×10^9
ResNet-152	60.2×10^6	11.3×10^9

ResNets are more popular than Inception as backbones for transfer learning because their generic, modular design is easier to adapt and modify.

Batch Normalization is Critical

Batch Normalization is applied after every convolution and before the activation. Without BN, training a 50+-layer ResNet is infeasible. Recent work (Huang et al., 2019) showed

that self-normalizing activations can substitute for BN and skip connections simultaneously, but BN + skip connections remains the standard recipe.

1.10 Pre-activation ResNet / ResNet-v2 (2016)

He et al. studied identity mapping variants and proposed moving BN and ReLU *before* the weight layers (pre-activation) rather than after. The identity shortcut then passes completely unmodified:

$$\mathbf{h} = \mathbf{x} + \mathcal{F}(\mathrm{BN}(\mathrm{ReLU}(\mathbf{x}))) \,.$$

On CIFAR-10, ResNet-1001 improves from 7.61% (original) to **4.92%** (pre-activation) with no parameter change, demonstrating that the exact placement of normalization matters for very deep networks.

1.11 ResNeXt (2016)

ResNeXt (Xie et al., 2016) achieved **3.0%** top-5 error at ILSVRC-2016 (second place; first was an ensemble at 2.9%). It introduces *grouped convolutions* controlled by a *cardinality* parameter C: instead of one large 3×3 convolution operating on all channels, C independent groups each operate on $1/C$ of the channels.

Grouped Convolution Parameter Count

For a 3×3 convolution with C_{in} input and C_{out} output channels split into C groups:

$$\text{Parameters} = C \times \frac{C_{\text{in}}}{C} \times 3^2 \times \frac{C_{\text{out}}}{C} = \frac{3^2\, C_{\text{in}}\, C_{\text{out}}}{C}.$$

Cardinality $C = 32$ reduces the 3×3 cost by $32\times$ while doubling the 1×1 costs, keeping the total roughly constant.

Grouped convolutions date back to AlexNet, where they were a GPU memory workaround. ResNeXt repurposed them as an intentional architectural design principle. The sparseness ratio of each block is $1 - 1/C$.

1.12 DenseNet (2016)

DenseNet (Huang et al., 2016) extends the skip-connection idea to its logical extreme: *every layer receives the concatenated feature maps of all preceding layers.* Layer l takes $\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{l-1}$ as input and produces an output of *growth rate* G channels.

Dense Block Connectivity

$$\mathbf{x}_l = H_l([\mathbf{x}_0,\, \mathbf{x}_1,\, \ldots,\, \mathbf{x}_{l-1}]) \,,$$

where $[\cdot]$ denotes channel-wise concatenation and $H_l : \mathbb{R}^{G(l-1)} \to \mathbb{R}^G$ is a BN→ReLU→Conv block.

Dense connectivity enables feature reuse, provides implicit deep supervision (gradients reach early layers directly through concatenation paths), and reduces overfitting. DenseNets achieve better top-1 accuracy than ResNets of comparable parameter count: DenseNet-169 beats ResNet-50 on ImageNet while using fewer parameters.

1.13 Squeeze-and-Excitation Network (SENet, 2017)

SENet (Hu et al., 2017) won ILSVRC-2017 — the final year — with **2.2%** top-5 error. It adds a lightweight *channel-attention* module to any existing block.

SE Block

Given a feature map $\mathbf{U} \in \mathbb{R}^{H \times W \times C}$:

1. **Squeeze**: Global average pooling $\rightarrow$ descriptor $\mathbf{z} \in \mathbb{R}^C$.
2. **Excitation**: $\mathbf{s} = \sigma(\mathbf{W}_2 \operatorname{ReLU}(\mathbf{W}_1 \mathbf{z}))$, with $\mathbf{W}_1 \in \mathbb{R}^{C/r \times C}$ and $\mathbf{W}_2 \in \mathbb{R}^{C \times C/r}$ (reduction ratio r, typically 16).
3. **Scale**: $\tilde{\mathbf{u}}_c = s_c \cdot \mathbf{u}_c$ (channel-wise scaling).

The SE block adds only $\frac{2C^2}{r}$ parameters but lets the network re-weight channels based on global image statistics. It can be attached to Inception modules (SE-Inception) or ResNet blocks (SE-ResNet) with minimal overhead.

1.14 MobileNet V2 (2018)

MobileNet V2 targets *mobile inference*: competitive accuracy with minimal multiply-accumulate operations. The core building block is the *inverted residual with linear bottleneck*.

1.14.1 Depthwise Separable Convolutions

A standard $R \times R$ convolution with C_l input and C_{l+1} output channels costs $R^2 C_l C_{l+1}$ parameters. *Depthwise separable convolutions* factor this into:

1. A *depthwise* $R \times R$ convolution applied independently per channel: $R^2 C_l$ params.
2. A *pointwise* 1×1 convolution mixing channels: $C_l C_{l+1}$ params.

Total: $R^2 C_l + C_l C_{l+1}$, vs. $R^2 C_l C_{l+1}$ — roughly $R^2\times$ cheaper. Depthwise separable convolutions are the extreme case of grouped convolutions (cardinality $= C_l$).

1.14.2 Inverted Residuals

Standard ResNet bottlenecks shrink channels, apply a 3×3, then expand. MobileNet V2 *inverts* this: expand (cheap 1×1) $\rightarrow$ depthwise 3×3 $\rightarrow$ project back to small dimension (1×1). The skip connection runs between the low-dimensional *bottleneck* representations, not across the high-dimensional expansion.

No ReLU after Bottleneck Projection

The final 1×1 projection uses a *linear* activation. Applying ReLU to a low-dimensional representation collapses the information irreversibly (it sets half the values to zero while the subspace has little redundancy).

1.15 EfficientNet (2019)

EfficientNet (Tan & Le, 2019) asks: given a fixed compute budget, how should one scale a CNN along depth d, width w, and input resolution r? The three dimensions are not independent: a higher-resolution image needs a deeper network (larger receptive field) and a wider one (more fine-grained features).

Compound Scaling

Let ϕ be a user-specified resource coefficient. Scale:

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi,$$

subject to $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$, with $\alpha, \beta, \gamma \geq 1$. Since FLOPs $\propto d \cdot w^2 \cdot r^2$, total FLOPs scale as 2^ϕ. Constants α, β, γ are found once by a small grid search on the baseline architecture (EfficientNet-B0); ϕ is then varied to obtain B1–B7.

The baseline B0 uses inverted residuals (MobileNet V2) with SE blocks (SENet). **EfficientNet-B7** achieves **84.4%** top-1 accuracy on ImageNet with 66M parameters — 8.4× smaller and 6.1× faster than the previous state-of-the-art GPipe. B0 already beats ResNet-50 with 4.9× fewer parameters and 11× fewer FLOPs.

1.16 Vision Transformer (ViT, 2020)

ViT (Dosovitskiy et al., 2020) applies a pure Transformer — no convolutions — to image classification by treating an image as a sequence of patches.

An image is divided into non-overlapping 16×16-pixel patches, each linearly projected to a d-dimensional embedding; a learnable `[CLS]` token is prepended and positional encodings are added. The sequence is processed by a standard Transformer encoder.

Self-Attention (single head)

Given input set $\mathbf{H}^{(0)} \in \mathbb{R}^{d \times T}$ (T patches):

$$\mathbf{Q}^{(l)} = \mathbf{W}_Q \mathbf{H}^{(l-1)}, \quad \mathbf{K}^{(l)} = \mathbf{W}_K \mathbf{H}^{(l-1)}, \quad \mathbf{V}^{(l)} = \mathbf{W}_V \mathbf{H}^{(l-1)},$$

$$\mathbf{H}^{(l)} = \operatorname{softmax}\left(\frac{\mathbf{Q}^{(l)} \mathbf{K}^{(l)^\top}}{\sqrt{d_k}}\right) \mathbf{V}^{(l)}.$$

The attention matrix is $T \times T$, so complexity is $\mathcal{O}(T^2)$ — quadratic in the number of patches (and prohibitive for full-resolution images).

ViT requires large-scale pre-training

Unlike CNNs, ViT lacks the spatial inductive biases (locality, translation equivariance) that make CNNs data-efficient. ViT only matches or beats CNNs when pre-trained on very large datasets (JFT-300M, 300 million images) and then fine-tuned on ImageNet.

1.17 Self-Supervised Learning (from ∼2020)

Self-supervised pre-training learns representations from *unlabeled* data using a *pretext task* that requires semantic understanding. Two main paradigms have emerged.

1.17.1 Contrastive Learning (e.g., SimCLR)

Two different augmentations of the same image form a *positive pair*; augmentations from different images form *negative pairs*. The network is trained to maximize similarity of positive pairs and minimize similarity of negatives.

InfoNCE Loss (NT-Xent)

$$\mathcal{L}\Big((\mathbf{h}_0, \mathbf{h}_j),\ \{\mathbf{h}_l\}_{l=1}^{L}\Big) = -\log \frac{\exp\big(\cos(\mathbf{h}_0, \mathbf{h}_j)/\tau\big)}{\sum_{l=1}^{L} \exp\big(\cos(\mathbf{h}_0, \mathbf{h}_l)/\tau\big)},$$

where $\cos(\mathbf{h}_i, \mathbf{h}_j) = \frac{\mathbf{h}_i^{\top}\mathbf{h}_j}{\|\mathbf{h}_i\|\|\mathbf{h}_j\|}$ is cosine similarity and $\tau > 0$ is a temperature hyperparameter.

1.17.2 Masked Image Modeling (e.g., MAE)

Random patches of an image are masked; the model must reconstruct them. Analogous to BERT's masked language modeling, this forces the network to learn global semantic structure.

1.18 Key Architectural Lessons

The drop from ∼26% to ∼2% top-5 error between 2012 and 2017 was driven by a small set of recurring design principles:

1. **Small kernels**: Stack 3×3 convolutions instead of large ones — same receptive field, more non-linearities, fewer parameters.
2. **Skip connections**: Highway gates or identity shortcuts enable gradient flow and training of very deep networks.
3. **Global Average Pooling**: Replace FC layers with GAP to eliminate spatial-dimension parameters and reduce overfitting.
4. 1×1 **convolutions**: Change channel count without altering spatial dimensions; enable bottlenecks.
5. **Bottleneck blocks**: Compress channels before costly 3×3 operations, simulating sparse connectivity on dense hardware.
6. **Grouped / depthwise convolutions**: Redistribute computation toward channel mixing; enable extreme parameter efficiency (MobileNet).

7. **Compound scaling**: Depth, width, and resolution are interdependent; balance them together for optimal efficiency (EfficientNet).
8. **Attention**: Channel-wise (SE) and spatial (Transformer) attention lets the network focus resources on the most informative parts of the input.

2

Semantic Segmentation

3

Object Detection and Localization

4

Autoencoders

5

Variational Autoencoders

6

GANs and W-GANs

7

Metrics (FID) and Normalizing Flows

8

Diffusion Probabilistic Models

9

Bayesian Deep Learning and Uncertainty

10

Graph Neural Networks

11

Theory